Cancellation coverage matrix — operation-scoped cancellation (`b317833`)

Candidate	**`b317833e876332b528759cf9b3ff546d1b20ba39`** on `fix/operation-scoped-cancellation`
Lineage	`f764c95` (B1 recovery scope + B6 killable subprocess) → `b317833` (B2/B3/B7 production-boundary tests + B7 test seam)
Base (production)	`origin/main` = `8c03b07` (untouched)
Purpose	Honest requirement→test→assertion map for every cancellation path, stating for each case **what level of realism** the test exercises (real Chromium child / real isolated subprocess / broker unit / in-process unit) and **what it does not prove**.

This document answers reviewer point 2. It is deliberately conservative: where a test proves *ordering* or *admission-gate* behaviour rather than *mid-flight interruption*, it says so, and the open gaps are listed explicitly in the final section rather than folded into a "covered" row.

Realism legend

Level	Meaning
R — real Chromium child	A real isolated job child (`runPipelineIsolated` → `job-child.ts`) drives the **production** browser call sites against a local HTTP server, with a real Chromium process observed via `/proc`.
S — real isolated subprocess	A real OS child process (pdf-parse worker via `runProcessCollect`/`extractViaPdfParseSubprocess`) is spawned and killed; the test observes real SIGKILL / reaping.
U(broker)	The real admission broker / in-flight map / scope machinery is exercised in-process (no network, no Chromium), driving the actual production functions (`acquireProcSlot`, `processDocument`, `runCompanyPdfRecovery`).
U(unit)	Pure in-process unit assertion over the scope/deadline primitives.

"Production code path" means the assertion runs through the same function production calls — not a re-implementation in the test.

---

1. Deadline / scope propagation (B1 — company PDF recovery)

Boundary: `runCompanyPdfRecovery` (pipeline.ts:275), called in production at pipeline.ts:1714 (default `fetchFn` = real `fetchIssuerPdfsWithPrimedSession`).

#	Requirement	Test :: subtest	Key asserti
1.1	One scope is armed and **passed to every fetch call** across all origins	`company-pdf-recovery-boundary.test.ts` :: B1-1	identical `sco` object observed at the fetch boundary fo every origin `scope was passed to th fetch boundary`)
1.2	Recovery budget is **capped by `min(PDF_RECOVERY_MAX_BUDGET_MS, remainingMs)`**	B1-2	armed deadline ≈ `remainingMs` when `remainingMs` cap
1.3	**No fetch is dispatched after the deadline**; the untried tail stays retryable	B1-3	origins after expiry are never passe to `fetchFn`; t URLs remai unattempte (retryable)
1.4	Attempt budget caps total attempts **across** origins (company-wide, not per-origin)	B1-4	total fetch attempts ≤ attemptBud summed across origi
1.5	Healthy path: all origins fetched, nothing filtered	B1-5	every origin attempted; r spurious ab
1.6	Default `fetchFn` routes through the **real admission boundary** and withdraws at the company deadline	B1-6	with the production default `fetchFn`, a queued wait withdraws when the company deadline elapses

Mutation proof (B1): removing scope propagation (`fetchFn(origin, slice, { outcomes, scope })` → `{ outcomes }`) makes **B1-1 fail** (`AssertionError: a scope was passed to the fetch boundary (was DEAD/omitted before B1)`); B1-2/B1-3 cascade. Reverted clean.

What B1 does NOT prove: B1 operates at the recovery-orchestration boundary, not against a live network fetch. It proves the scope is *constructed, capped, and passed*, and that dispatch stops after the deadline — it does not prove that an HTTP fetch already in flight is torn down mid-transfer (that is the active-interruption gap, §6).

---

2. Process-admission cancellation (B2 — `acquireProcSlot`)

Boundary: `acquireProcSlot` local-broker queue (processor.ts:618-644), abort listener at 641.

#	Requirement	Test :: subtest	Key assertion	Lev
2.1	A **queued** proc waiter whose scope expires **withdraws** and is never granted a slot	`cancellable-proc-admission-boundary.test.ts` :: B2-1	waiter settles `withdrawn`, not `hung`; no slot consumed	U(bro
2.2	The next waiter is admitted promptly once the holder releases	B2-2	next waiter granted; `active` returns to expected	U(bro
2.3	Grant/abort race (abort **before** release): waiter withdraws, slot returns cleanly — no leak, no phantom grant	B2-3	`active` drains to 0; no double-grant	U(bro
2.4	A **pre-aborted** scope is refused at admission and consumes no slot	B2-4	throws `OperationAbortedError`; `active` unchanged	U(bro

Mutation proof (B2): removing the queued-waiter abort listener (`if (sig) sig.addEventListener("abort", onAbort…)` at line 641) makes **B2-1 fail** (`'hung' !== 'settled'` — the waiter never withdraws), and B2-2/B2-3/B2-4 cascade with `proc slot leaked (active=1)`. Reverted clean.

---

3. Stalled-extraction cancellation (B3 — `runProcessCollect` subprocess)

Boundary: `runProcessCollect` abort registration (processor.ts:747-750), SIGKILL in `onAbort` (721-725).

#	Requirement	Test :: subtest	Key assertion	Le
3.1	A **hung extraction child** is SIGKILLed on scope abort; parent stays responsive; slot restored	`stalled-extraction-boundary.test.ts` :: B3	parent settles **promptly after abort** (elapsed ≪ child timeout); slot restored	**S**
3.2	`extractViaPdfParseSubprocess` throws **retryable** on an already-aborted scope and takes **no** slot	B3/B2	throws; `active` unchanged	**S**

Mutation proof (B3): disabling the abort registration block makes **B3 fail** with `parent settled promptly after abort (elapsed=60005ms)` — the child runs the **full 60 s timeout** instead of being killed at abort. Reverted clean. This is the clearest active-interruption proof in the suite: a real child process is really killed at the abort instant.

---

4. In-flight coalescing cancellation (B7 — `processDocument`)

Boundary: `processDocument` in-flight dedup + liveness guard (processor.ts:2519).

#	Requirement	Test :: subtest	Key assertion	Level
4.1	Concurrent same-URL callers **coalesce to one** underlying fetch	`inflight-coalescing-boundary.test.ts` :: B7-1	exactly 1 underlying fetch for 2 concurrent callers	U(broker)
4.2	A new caller does **NOT** coalesce onto an in-flight entry whose scope has **aborted** — it starts its own attempt	B7-2	2 underlying fetches (aborted entry not reused)	U(broker)

Mutation proof (B7), two mutants: - Disable coalescing (`if (false && existing …)`) → **B7-1 fails** (`2 !== 1`). - Drop the liveness check (`if (existing) return …`, ignoring `scope.aborted()`) → **B7-2 fails** (`1 !== 2` — the new caller wrongly coalesces onto the aborted entry). Both reverted clean. Two independent mutants are needed because the two clauses of the guard are independently load-bearing.

---

5. Browser lifecycle close/reap ordering (real Chromium) — `browser-close-order.test.ts`

Boundary: real isolated job children driving **production** browser call sites (`fetchPdfViaBrowser`, `fetchIssuerPdfsWithPrimedSession`), `MAX_CONCURRENT_BROWSER=1`, live Chromium observed via `/proc` at every permit grant/return. Invariant asserted on **every** exit path: **work stopped → Chromium closed → termination confirmed → permit returned**.

#	Requirement	Test :: subtest	Key assertion	Leve
5.1	**Normal op:** marker extracted; Chromium gone before the permit returns	success path	`r.ok`, marker matched, `liveChromium == []` at release	**R**
5.2	**Fetch failure** (connection reset; direct + pdfRecovery): close before release	fetch-failure path	no text yielded; `liveChromium == []` at both releases	**R**
5.3	**Cancellation — admission gates:** a caller refused **after** acquiring the permit, and one refused **before** acquiring, both raise `BrowserUnavailableError: lifecycle cancelled`; permits return with Chromium closed	cancellation path (B_postAcquire, C_preAcquire)	both error strings match; `liveChromium == []` at release	**R**
5.4	**Cancellation — in-flight:** a fetch already holding the permit at the cancel instant **completes**, then Chromium closes before the permit returns	cancellation path (A_inflight)	`A_inflight.ok == true`; `liveChromium == []` at release	**R**
5.5	**Child death:** pipeline deadline → SIGKILL while holding the permit → parent group reap before `releaseAll`	child-death path	job fails; `liveChromium == []` at release; broker drained	**R**

Mutation proof (ordering): documented in the prior browser-lifetime review — releasing the permit *before* closing Chromium makes these tests fail (the `/proc` scan sees a live Chromium at release). Not re-run here (no code in that path changed between that release and `b317833`).

What §5 proves and what it does NOT: - It **proves** the close/reap **ordering invariant** on all four exit paths with a **real Chromium child**, and it proves cancellation is honoured at the **admission gates** (post-acquire and pre-acquire refusal — 5.3). - It **does NOT prove instantaneous interruption of an in-progress CDP operation.** Row 5.4 is explicit and honest: the in-flight fetch **runs to completion** and cancellation takes effect at the **next gate**, not mid-navigation. A `page.goto`/`page.evaluate`/WAF-challenge already executing inside Chromium is not torn down at the abort instant by this layer; the deadline is enforced *around* it (admission refusal for new work, and ultimately the job-isolation SIGKILL, §5.5).

---

6. Honest gaps — requirements NOT fully proven by behavioural test

These are stated so the reviewer does not read a green suite as more than it is.

1. **Active in-page CDP interruption is not instantaneous.** An HTTP fetch or Chromium navigation/`page.evaluate` already in flight when the scope aborts is not cancelled mid-operation by the scope machinery. Cancellation is enforced at the **next admission gate** (B2/B7 refusal), by **dispatch suppression** (B1-3: no new fetch after deadline), and — as the hard backstop — by the **job-isolation SIGKILL** of the whole child (observed in 5.5). The in-flight operation itself can run to its own completion or its own timeout first. Row 5.4 demonstrates exactly this, by design.
2. **Deadline propagation into the browser fetch is bounded, not threaded.** B1 proves the scope is *passed* to the fetch boundary and that *dispatch* stops after the deadline; it does not prove a per-request network timeout equal to `remainingMs` is applied inside `fetchIssuerPdfsWithPrimedSession`. The effective ceiling on an already-started browser fetch is the pipeline deadline → SIGKILL, not a scope-derived socket timeout.
3. **Production recovery scope degenerates to the constant.** The production call (pipeline.ts:1714) omits `remainingMs`, so in production the recovery scope's deadline is `PDF_RECOVERY_MAX_BUDGET_MS` (600000) rather than a per-company remaining budget; the company deadline is enforced externally by job-isolation SIGKILL. Tests supply `remainingMs` to assert the capping logic (B1-2), but that narrower cap is **not** what runs in production today. This is a semantic point the reviewer should weigh.
4. **Specific production stalls remain unobserved.** The Idemitsu ~18-minute single-step and the AXA analysis-prep ~7m26s stall (see `TIMEOUT_INVESTIGATION.md`) are not reproduced by any test here; they are out of scope for this cancellation patch and are not claimed fixed.

Summary

Boundary	Behavioural test	Mutation-fail demo	Realism
B1 deadline/scope propagation	6 subtests (B1-1..B1-6)	scope-drop → B1-1 fail	U(broker)
B2 proc admission	4 subtests (B2-1..B2-4)	listener-removed → B2-1 `hung`	U(broker)
B3 stalled extraction	2 subtests	abort-reg-disabled → full 60 s	**S (real kill)**
B6 killable extraction	2 subtests	(size-cap → subprocess; see REVIEW_PACKAGE)	**S (real subprocess)**
B7 in-flight coalescing	2 subtests (B7-1, B7-2)	2 mutants → B7-1 / B7-2 fail	U(broker)
Browser close/reap ordering	4 paths	release-first → ordering fail	**R (real Chromium)**

All 7 files green on `b317833` (27 subtests). Every mutation in this document was applied, observed to fail the named assertion, and reverted to a clean tree **in this session**. The gaps in §6 are real and are the honest boundary of what the suite proves.